

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 713 — Software Security

Why code has vulnerabilities, and what actually finds them

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 9 (with CSCE 701 and CSCE 711)

Prerequisites: CSCE 611 Operating Systems for the memory model, CSCE 605 Compiler Design for the analysis material, and CSCE 627 Theory of Computability for what analysis cannot promise; CSCE 606 Software Engineering for the process

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A security assessment of a real codebase — a fuzzing campaign with its coverage reported, a static analysis run with its findings triaged, and every defect you found fixed, with a written account of what the tooling could not have found

Course description

This is a defensive course about defects. It covers **why vulnerabilities exist, which classes account for the severe ones, and which techniques actually find them** — so that you can build software with fewer and find the ones you have. **The exercises are performed on your own code and on deliberately vulnerable training targets**, which Module 01 §4 states as a non-negotiable constraint.

The organising claim is that **vulnerabilities are ordinary bugs with an adversary attached. A buffer overflow is a correctness defect that happens to be reachable by someone who wants it** — which means the techniques that find correctness defects find vulnerabilities, and the difference is in what you consider an input and who you imagine supplying it (Module 01 §2).

The second theme is that **memory safety dominates the severe-defect data, and that it is a language property rather than a discipline problem. Decades of careful C produced a steady supply of the same defect classes** (Modules 02 and 03), which is **an argument about tools rather than about programmers**, and Module 12 covers what to do when rewriting is not available.

The third is that **the tooling has hard theoretical limits and is still worth running. CSCE 627 established that non-trivial program properties are undecidable**, so every analyser must choose between false positives and false negatives — **and Modules 07 through 09 are about choosing deliberately and reading the output honestly**.

And the closing position is about honest claims. **Module 13 is about what a fuzzing campaign, a static analysis run, and a code review each establish** — because **'the scanner found nothing'** is a statement **about the scanner**, and saying what you examined, with what, for how long, is the claim that can be

checked.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why vulnerabilities exist and what makes one severe.
- 2 Explain the memory safety defect classes and their mechanisms.
- 3 Explain integer and type confusion defects.
- 4 Explain injection as a parsing problem and fix it structurally.
- 5 Identify concurrency defects with security consequences.
- 6 Explain deserialisation and trust boundary failures.
- 7 Use static analysis and triage its output.
- 8 Run a fuzzing campaign and measure it properly.
- 9 Use sanitisers and runtime detection effectively.
- 10 Integrate security into a development lifecycle.
- 11 Review code for security with a systematic method.
- 12 Explain exploit mitigations and what each costs.
- 13 State what an assessment of code honestly establishes.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Why Software Has Vulnerabilities	What makes a bug a vulnerability?
2	Memory Safety	What exactly goes wrong, at the machine level?
3	Integers and Types	Why did the length check pass?
4	Injection and Parsing	Why does escaping keep failing?
5	Concurrency Defects	Which races are vulnerabilities?
6	Deserialisation and Trust Boundaries	What happens when you deserialise untrusted data?
7	Static Analysis	Why does the analyser report things that are not bugs?
8	Fuzzing	How do you generate inputs that reach the interesting code?

Wk	Module	Core question it answers
9	Sanitisers and Runtime Detection	The test passed. Was the program actually correct?
10	The Secure Development Lifecycle	Which practices actually reduce defects?
11	Code Review for Security	What do you read, and in what order?
12	Exploit Mitigations	The bug exists. Why is it hard to exploit?
13	Claiming Code Security Honestly	The tools found nothing. What does that tell anyone?

Projects

Project 1 — A defect class, understood completely (after Module 6)

Take one defect class, build a minimal vulnerable program, and then make it impossible.

Requirements

- **One class chosen from Modules 02 through 06**, with its CWE identifier and its mechanism explained in your own words.
- **A minimal program exhibiting it** — yours, under fifty lines, with the defect reachable from an input.
- **A demonstration that it is a defect**: a sanitiser report, a crash, or a wrong answer, captured.
- **Three fixes at different levels**: the local fix, the API-level fix that prevents recurrence, and the language or tooling fix that eliminates the class.
- **And the detection analysis**: which of static analysis, fuzzing, sanitisers, and review would have found it, and which would not, with each claim tested.

Done means

- **The mechanism explained correctly**, at the level of what the machine actually does.
- **Three fixes at genuinely different levels — the API-level fix is the one that demonstrates understanding**, and it is graded hardest.
- **The detection claims tested rather than asserted**: you ran the tools and reported what happened, including where your prediction was wrong.
- **And the class-elimination argument honest about its cost**, because 'rewrite it in a safe language' has a price that Module 12 asks you to state.

Project 2 — A real codebase, assessed (after Module 12)

Assess a real open-source project you use, with fuzzing and static analysis, and fix what you find.

Requirements

- **A fuzzing campaign with a harness you wrote**, run for a stated duration, **with coverage reported** (Module 08 §3).
- **A static analysis run with every finding triaged** — true positive, false positive, or undetermined, with reasons (Module 07 §3).
- **A manual review of the highest-risk component**, chosen and justified by Module 11's method.
- **Every genuine defect reported responsibly and fixed where you can**, with the patch offered upstream.
- **Coverage and limitation analysis**: what fraction of the code your fuzzing reached, and what your methods could not have found.
- **And an honest claim** in Module 13 §1's form.

Done means

- **The harness non-trivial** — it reaches real parsing logic rather than an input validation function, and the coverage figure demonstrates it.
- **Every static finding triaged with a reason; 'probably a false positive' is not a triage** and is graded as an untriaged finding.
- **Anything found reported responsibly**, which is assessed as a professional obligation rather than a courtesy.
- **And the limitation analysis specific: naming the defect classes your campaign structurally could not have found is the deliverable**, and it is what separates an assessment from a tool run.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Dowd, McDonald & Schuh — The Art of Software Security Assessment	The primary reference for Modules 02 through 06 and 11. Dated in its examples and unmatched in its treatment of how defects arise and how to find them by reading. Library copy.
Zeller et al. — The Fuzzing Book (free, interactive)	Module 08's primary source, free and executable. Builds fuzzers from scratch in the browser, which is the right way to understand what coverage guidance actually does.
Møller & Schwartzbach — Static Program Analysis (free PDF)	Module 07's theory, free. The lattice and fixpoint material from CSCE 605 applied to defect detection, and honest about the precision limits.

Resource	Role
Google — Building Secure and Reliable Systems, the development chapters (free PDF)	Modules 10 and 12, free in full — and notable for arguing that secure defaults in frameworks beat developer education, which is this course's position too.
The CWE and the OWASP Top Ten (free)	The shared vocabulary for Modules 02 through 06. Use CWE identifiers when reporting; it makes a finding searchable and comparable.
Erickson — Hacking: The Art of Exploitation, 2nd edition	Optional, for Module 12's mitigations. Understanding why a mitigation works requires understanding what it stops — and the exercises are for your own machine and the book's own targets only.

Tooling

- **Your own code, and deliberately vulnerable training targets. Nothing else, ever** — which is Module 01 §4's constraint, matches CSCE 701's, and is not negotiable.
- **A C or C++ toolchain with sanitisers: clang with AddressSanitizer, UndefinedBehaviorSanitizer, and ThreadSanitizer** — free, and Module 09's whole subject.
- **A coverage-guided fuzzer: libFuzzer, AFL++, or cargo-fuzz** for Module 08, plus atheris or jazzier if you work in a managed language.
- **Static analysers: clang-tidy and the Clang Static Analyzer, plus CodeQL or Semgrep** for writing your own queries in Module 07 §4.
- **A deliberately vulnerable target set: OWASP Juice Shop or WebGoat for the web classes, and google/fuzzer-test-suite for memory defects. Free, legal, and designed for exactly this.**
- **And a real open-source project you use**, to fuzz and analyse for Project 2. **Report what you find responsibly**, which Module 13 §2 covers as a professional obligation.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.